Deep Learning for 3D Computer Vision - Final Project

Retrieval-Augmented Robotic Manipulation with Pretrained 3D Representations

Abstract

This project studies whether geometric similarity between robotic manipulation scenes provides a useful retrieval signal for reusing past experiences. We build an RLBench experience database and compare random, pose, image/color, geometric, Uni3D, and Point Transformer V3 retrieval. The primary evaluation contains 19 tasks and 1,804 episodes. We also evaluate robustness to viewpoint, occlusion, and geometry noise, and test a frozen-backbone trajectory-state projection head with a held-out test-to-train protocol. The results show that 3D retrieval is meaningfully better than random retrieval, but color and appearance baselines are stronger in this dataset. Uni3D is stronger than PTv3 in the main results. The projection head does not improve held-out performance over original Uni3D. The simulator experiment validates integration but is not a learned planning benchmark because the exported episodes do not contain explicit action arrays.

The complete implementation, reproduction instructions, and supporting scripts are available in the project repository: RAG for Robotics on GitHub.

1. Introduction and Related Work

1.1 Motivation and Research Question

Retrieval-augmented robotics uses a memory of previous robot experiences to support decisions in a new scene. Each memory item can be represented as a tuple containing a scene observation, a trajectory, and outcome metadata. Given a new point-cloud observation, the system retrieves nearby experiences and can pass their trajectories to a downstream policy or planner.

The research question is:

Does 3D geometric similarity between scenes, encoded via a pretrained 3D
representation, provide a meaningful retrieval signal for robotic
manipulation planning, and under what scene variations does this signal remain
reliable?

The initial hypothesis was that geometric representations would be more robust than appearance-based representations under viewpoint, occlusion, and object variation. The project therefore treats representation quality and similarity as the primary experimental variables. A learned action-generating planner was part of the proposed long-term direction, but the completed implementation focuses on retrieval and offline downstream evidence.

1.2 Related Work

RLBench was introduced by James et al. as a robot-learning benchmark and simulation environment containing diverse manipulation tasks and demonstrations. It is the source of the simulated manipulation episodes used here. The present work differs from a policy-learning benchmark because the main metric evaluates whether scene embeddings retrieve episodes from the same task group.

Mandlekar et al. introduced robomimic as a reproducible framework for learning from offline robot demonstrations and showed that demonstration quality and algorithmic choices strongly affect manipulation learning. This motivates our explicit manifest, fixed splits, and artifact-based evaluation. Our focus is earlier in the pipeline: selecting useful demonstrations before training a control policy.

MimicGen studies scalable generation of robot-learning demonstrations by adapting source demonstrations to new contexts. This is relevant future work for our system, because retrieved trajectories could eventually be transformed

before execution. Our current system retrieves trajectories but does not adapt them into new executable plans.

Kuroki et al. present retrieval-augmented policy training using a database of cooperative robot behaviors. Their work is a close conceptual precedent for using memory retrieval to support robot control. Our contribution is narrower: we isolate the question of whether pretrained 3D scene representations are useful retrieval keys, without claiming their policy-training or real-robot success setting.

Liu et al. study policies augmented with motion-planner supervision. This work highlights the distinction between a retrieval module and an action-generating policy. Our simulator pilot validates execution infrastructure and replay only; it does not implement a learned policy of this kind.

Uni3D, by Zhou et al., is a pretrained 3D foundation model that aligns point cloud features with image-text aligned representations. We use an official Uni3D checkpoint as a learned 3D retrieval encoder, while evaluating it on robotic manipulation episodes rather than its original object-level benchmarks.

Point Transformer V3, by Wu et al., emphasizes scalable serialized point-cloud processing and efficient large receptive fields. We use the Pointcept v1.5.2 runtime and a compatible pretrained checkpoint as a second modern 3D encoder. The comparison tests whether a modern point-cloud backbone automatically gives the best task-retrieval signal. The results suggest that this depends on the dataset and the task-relevance definition.

1.3 Positioning

The project combines a robotic experience database, pretrained 3D embeddings, nearest-neighbor retrieval, controlled perturbation tests, and an offline trajectory-transfer proxy. It does not claim to deliver a complete learned planner. This distinction is important because retrieval quality and planning success are related but different measurements.

1.4 Proposal Scope and Deliverables

The approved proposal asked whether pretrained 3D geometric representations can retrieve useful manipulation experiences, whether they remain reliable under scene variations, and whether retrieved experiences can support downstream planning. The completed scope delivers the first two parts rigorously and provides offline downstream and simulator-integration evidence for the third. The project deliberately reports the learned-planner component as future work rather than presenting replay as a learned planning result.

2. Method

2.1 Dataset and Experience Database

The full exported dataset contains 19 RLBench tasks and 1,804 episodes. The task set is: reach_target, open_drawer, slide_block_to_color_target, sweep_to_dustpan_of_size, meat_off_grill, turn_tap, put_item_in_drawer, close_jar, reach_and_drag, stack_blocks, light_bulb_in, put_money_in_safe, place_wine_at_rack_location, put_groceries_in_cupboard, place_shape_in_shape_sorter, push_buttons, insert_onto_square_peg, stack_cups, and place_cups.

Each episode has a manifest record, observation arrays, trajectory data, task identity, and metadata. Relevance annotations are task-group based: an episode is relevant when it belongs to the same task group as the query. This is a well-defined retrieval protocol, but it is not equivalent to measuring whether the retrieved trajectory is executable in a new scene.

In the terminology of this report, a **task** is a category of manipulation problem, such as opening a drawer. An **episode** is one recorded attempt at one task instance, from reset to termination. Thus, one task can contain many episodes. A **query** is the episode whose observation is used to search the database; a **candidate** is an episode eligible to be retrieved. The query is not itself counted as a candidate in leave-one-out evaluation.

2.1.1 Task Definitions

The full benchmark contains the following 19 RLBench task categories:

Task	Plain-language meaning
`reach_target`	Move the robot end-effector to a target location.

Task	Plain-language meaning
`open_drawer`	Pull open a drawer using its handle.
`slide_block_to_color_target`	Slide a block to a target with the matching color.
`sweep_to_dustpan_of_size`	Sweep material into a dustpan of the requested size.
`meat_off_grill`	Remove a piece of meat from a grill.
`turn_tap`	Rotate a tap handle to the required state.
`put_item_in_drawer`	Pick up an item and place it inside a drawer.
`close_jar`	Close a jar by placing or rotating its lid.
`reach_and_drag`	Reach an object and drag it to a target region.
`stack_blocks`	Place blocks on one another in a stack.
`light_bulb_in`	Insert a light bulb into its socket.
`put_money_in_safe`	Place money inside a safe.
`place_wine_at_rack_location`	Place a wine bottle at a specified rack location.
`put_groceries_in_cupboard`	Place grocery items inside a cupboard.
`place_shape_in_shape_sorter`	Insert a shape into the matching sorter opening.
`push_buttons`	Press the required buttons.
`insert_onto_square_peg`	Insert an object onto a square peg.
`stack_cups`	Stack cups in the required arrangement.
`place_cups`	Place cups at the required locations.

The task name is used to define relevance groups in the main evaluation. The episode identifier distinguishes separate attempts within a task, and the variation identifier records a task variation when available.

2.1.2 Robotic Experience Database

The experience database is an episode-indexed collection of scene observations, trajectories, and metadata. It is not a text-only RAG index: the retrieval key is computed from 3D or state observations, while the retrieved record may also contain the trajectory that could be used by a downstream controller.

Database component	Typical contents	Role in this project
Observation	$o_t = (I_t, D_t, P_t, C_t)$: RGB, depth, point cloud, camera metadata	Describes what the robot observed.
3D representation	$P_t = \{(x_j, y_j, z_j, r_j, g_j, b_j)\}_{j=1}^{M} \rightarrow z_e$	Provides the retrieval key for Uni3D, PTv3, and geometry baselines.
Trajectory	$\tau_e = \{(q_t, \dot{q}_t, g_t, p_t)\}_{t=1}^{T_e}$	Stores what the robot did during the episode.
Outcome metadata	$m_e = (y_e, \ell_e, v_e, s_e)$: success, task, variation, source	Supports grouping, filtering, and analysis.
Manifest row	$r_e = (id_e, paths_e, split_e, hash_e)$	Provides a reproducible index into the database.

The exported layout separates `observation.npz`, `trajectory.npz`, and `metadata.json`, with one row in `manifest.parquet` per episode. In the full dataset, the database contains 1,804 episodes over 19 tasks; the held-out projection experiment uses 422 training episodes and 141 test episodes.

2.1.3 Code Structure

The repository is organized around a small, reproducible experiment pipeline:

```
RAG_for_Robotics/
|-- src/action_retrieval/
```

```
|   |-- data/             Dataset schema, export, transforms, validation
|   |-- retrieval/        Episode loading, encoders, embeddings, ranking
|   |-- evaluation/       Retrieval metrics and evaluation protocol
|   |-- simulation/       RLBench source import and trajectory utilities
|   |-- downstream/       Offline downstream transfer interfaces
|   |-- visualization/    Visualization package namespace
|   `-- utils/            Environment and reproducibility helpers
|-- scripts/
|   |-- evaluate_retrieval.py             Main retrieval evaluation
|   |-- evaluate_robustness.py             Viewpoint/occlusion/noise tests
|   |-- train_action_aware_projection.py  Frozen-backbone projection head
|   |-- evaluate_projected_embeddings.py  Held-out projection evaluation
|   |-- evaluate_downstream_proxy.py       Offline trajectory-transfer proxy
|   |-- run_rlbench_planning_pilot.py     Guarded simulator replay pilot
|   `-- plot_report_figures.py             Report figure generation
|-- configs/              Dataset, encoder, retrieval, and experiment settings
|-- slurm/                Cluster jobs for setup, evaluation, and diagnostics
|-- tests/                Unit and integration tests for project code
|-- third_party/          External RLBench/PyRep integration boundary
|-- requirements.txt      General Python dependencies
|-- requirements-cluster.txt Validated Cluster additions and versions
|-- ENVIRONMENT_REPRODUCTION.md  Exact environment and checkpoint record
|-- README.md             Reproduction instructions and attribution
`-- FINAL_REPORT.md       This report
```

The `src/` package contains reusable project logic, `scripts/` contains experiment entry points, `configs/` makes settings explicit, and `slurm/` encodes the Cluster execution environment. Large datasets, model checkpoints, and generated result archives are kept outside the Git source tree and are referenced by their recorded paths and hashes.

2.2 Retrieval Encoders

Method	Representation
random	Random candidate ranking
pose_descriptor	Compact pose and state descriptor
rgb_histogram	Global RGB histogram
global_color	Global color statistics
geometry_only	Point-cloud geometry descriptor without color
uni3d	Official pretrained Uni3D point-cloud encoder
ptv3	Official Pointcept PTv3 runtime and pretrained checkpoint

All methods use the same episode database, relevance annotations, and ranking metrics. The retrieval cutoffs are k=1, 2, and 3.

2.3 3D Geometric Representations

For an episode e, the encoder receives a fixed-size representation of its point-cloud observations and returns a vector z_e. The exact network differs by method, but the retrieval interface is the same: encode every database candidate once, encode the query, compare vectors, and rank candidates by similarity.

Representation	Input	What it captures	Status
`geometry_only`	XYZ point samples	Coarse spatial shape and arrangement without color	Project baseline
`rgb_histogram`	RGB observations	Global color distribution	Project baseline
`global_color`	RGB summary statistics	Compact appearance and color cues	Project baseline
`pose_descriptor`	Compact state/pose features	Robot and object configuration cues	Project baseline
`Uni3D`	XYZRGB point cloud	Pretrained learned 3D foundation representation	Real pretrained backend
`PTv3`	Point-cloud coordinates and features	Pretrained serialized point-transformer features	Real Pointcept backend

Uni3D and PTv3 are external pretrained model runtimes. The project code provides their adapters, input conversion, checkpoint loading, device selection, and common embedding interface; it does not claim authorship of the upstream architectures or weights.

2.4 Retrieval, Offline Downstream, and Simulator Evidence

These are three different levels of evidence and should not be conflated:

Level	Operation	What it demonstrates	What it does not demonstrate
Retrieval	Rank stored candidate episodes for a query	Whether the representation retrieves same-task episodes	That a retrieved trajectory is executable
Offline downstream	Transfer or compare retrieved trajectory signatures without launching simulation	Whether retrieval can provide a useful downstream signal	Closed-loop control or planning success
Simulator integration	Launch CoppeliaSim through PyRep/RLBench and replay stored states/actions	Whether the software pipeline can initialize and execute a pilot	A learned planner, especially when actions are derived from state

CoppeliaSim is the simulator process. **PyRep** is the Python interface that connects code to CoppeliaSim, and **RLBench** supplies manipulation tasks, scene variations, demonstrations, and task APIs on top of that simulator. In our pilot, headless OpenGL initialization succeeded, but the available export contained derived joint-position replay rather than explicit ground-truth action arrays. The resulting 0/4 success rate is therefore an integration diagnostic, not a learned-planning result.

2.5 Metrics

For each query, candidates are ranked by embedding similarity. We report precision@k, recall@k, mean reciprocal rank (MRR), mean average precision@k (MAP@k), normalized discounted cumulative gain (NDCG@k), Top-1 accuracy, mean similarity scores, median first relevant rank, and hit rate@k.

2.6 Action-Aware Projection Head

The projection-head pilot freezes the Uni3D backbone and trains a small MLP to predict a compact trajectory-state signature derived from gripper pose. The MLP is not an action-generating planner and does not fine-tune Uni3D. Training uses 422 episodes, while held-out evaluation uses 141 test queries against 422 train candidates, with zero train/test overlap.

2.7 Software Environment

The experiments required a larger native and scientific Python stack than the retrieval scripts alone. Important components were PyTorch with the validated CUDA build, torchvision, torch-geometric, torch-scatter, torch-sparse, torch-cluster, spconv-cu124, cumm, timm, open3d, pyarrow, fastparquet, scipy, scikit-learn, matplotlib, and easydict. The simulator pilot also required gymnasium, cffi, natsort, PyRep, RLBench, and CoppeliaSim. These dependencies are separated between `requirements.txt` and `requirements-cluster.txt`; exact installed versions and checkpoint hashes are recorded in the reproducibility artifacts.

2.8 Robustness Protocol

Robustness perturbations are applied to query observations while the candidate database remains clean. The tested conditions are viewpoint rotation, partial occlusion, and geometry noise. This is a controlled proxy for scene variation; it does not replace evaluation on independently generated object geometries.

2.9 Implementation Ownership and External Code

The experiment orchestration and evaluation code in this repository was written for this project. This includes dataset export and validation, manifest and annotation handling, retrieval encoders and ranking, evaluation metrics, robustness perturbations, projection-head training/evaluation, reproducibility scripts, SLURM wrappers, and the simulator pilot harness.

Uni3D and Pointcept/PTv3 are external research codebases used as model runtimes. Their pretrained checkpoints and native dependencies were not written as part of this project. They are loaded through adapters in this repository, with configuration, checkpoint paths, key remapping, device selection, and fallback handling implemented here. RLBench, PyRep, CoppeliaSim, PyTorch, PyG, spconv, cumm, and other packages are also external dependencies. Their licenses and upstream references should be preserved in the submitted code folder.

The scientific contribution of this implementation is the common retrieval interface, controlled comparison protocol, task-group relevance evaluation, robustness setup, held-out projection-head experiment, and analysis of the resulting evidence. We do not claim authorship of the upstream model architectures or pretrained weights.

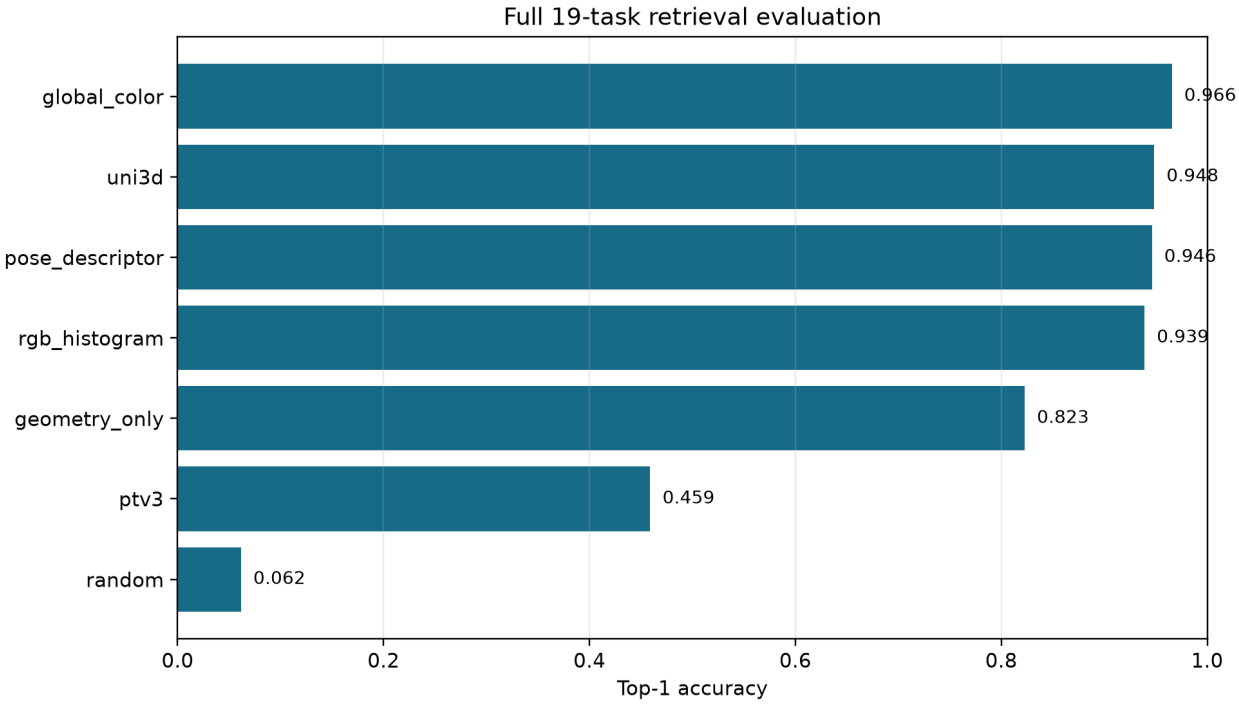
3. Evaluation and Results

3.1 Full 19-Task Retrieval

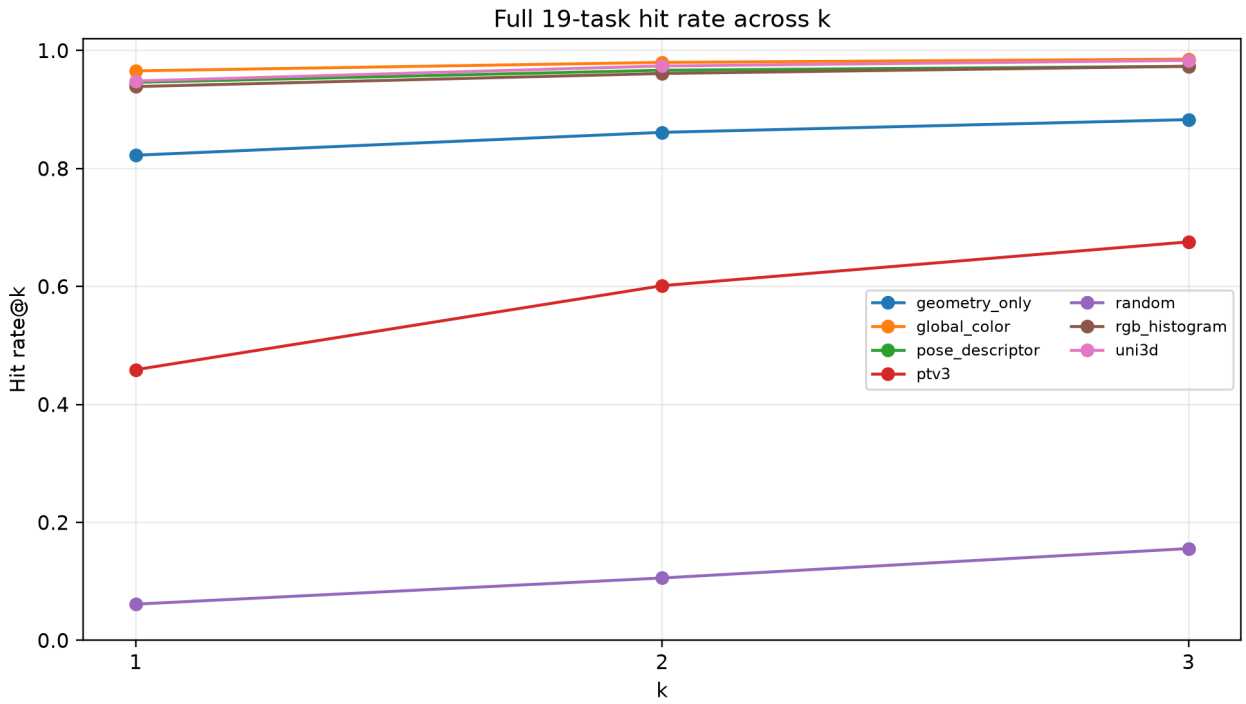
The primary results are stored in `retrieval_full/v2_multitask_full/summary_metrics.csv` and `retrieval_full/v2_multitask_full/evaluation.json`.

Method	Top-1 accuracy	Hit rate@3	MRR@3
random	0.0615	0.1558	0.1003
pose_descriptor	0.9462	0.9734	0.9587
rgb_histogram	0.9390	0.9734	0.9542
global_color	0.9656	0.9856	0.9747
geometry_only	0.8226	0.8830	0.8492
uni3d	0.9484	0.9834	0.9643
ptv3	0.4590	0.6757	0.5550

The full-dataset result shows that all non-random methods provide a meaningful task retrieval signal. Global color is strongest by Top-1 accuracy, followed by Uni3D and pose_descriptor. Geometry-only retrieval is substantially above random but below the color-aware methods. PTv3 is above random but weaker than the other tested representations in this configuration.



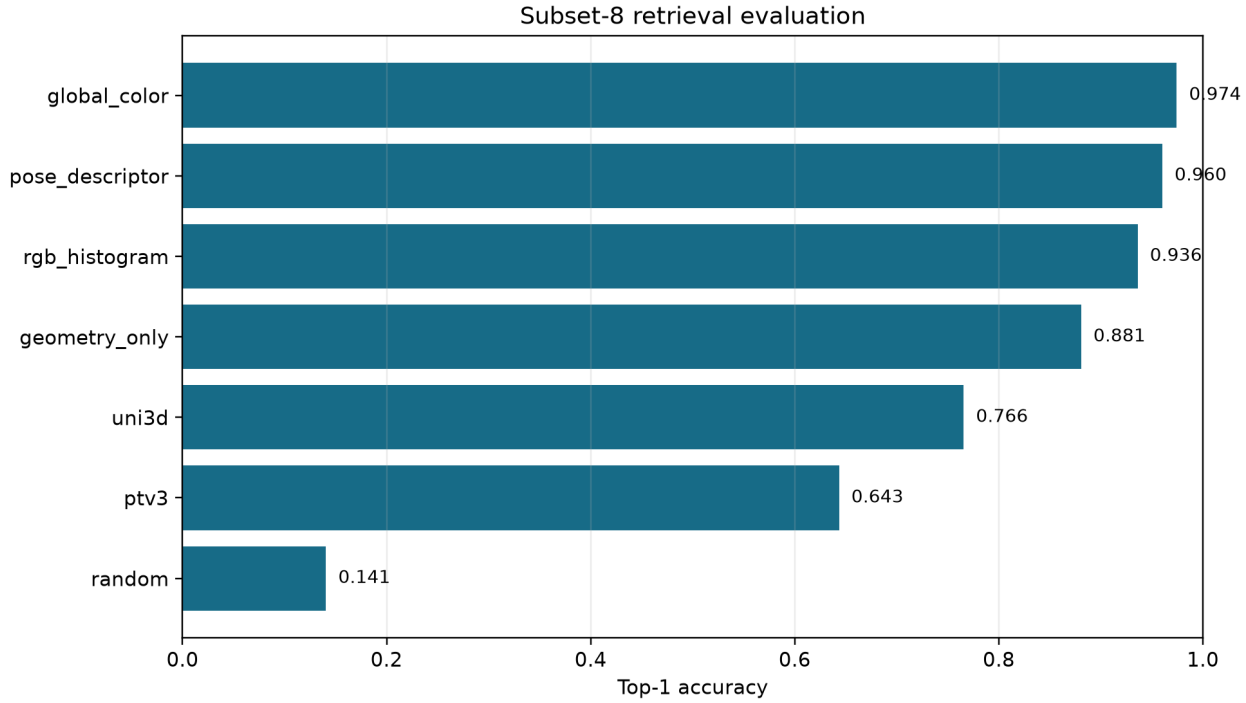
Full Top-1 comparison



Full hit rate by k

3.2 Subset-8 Retrieval

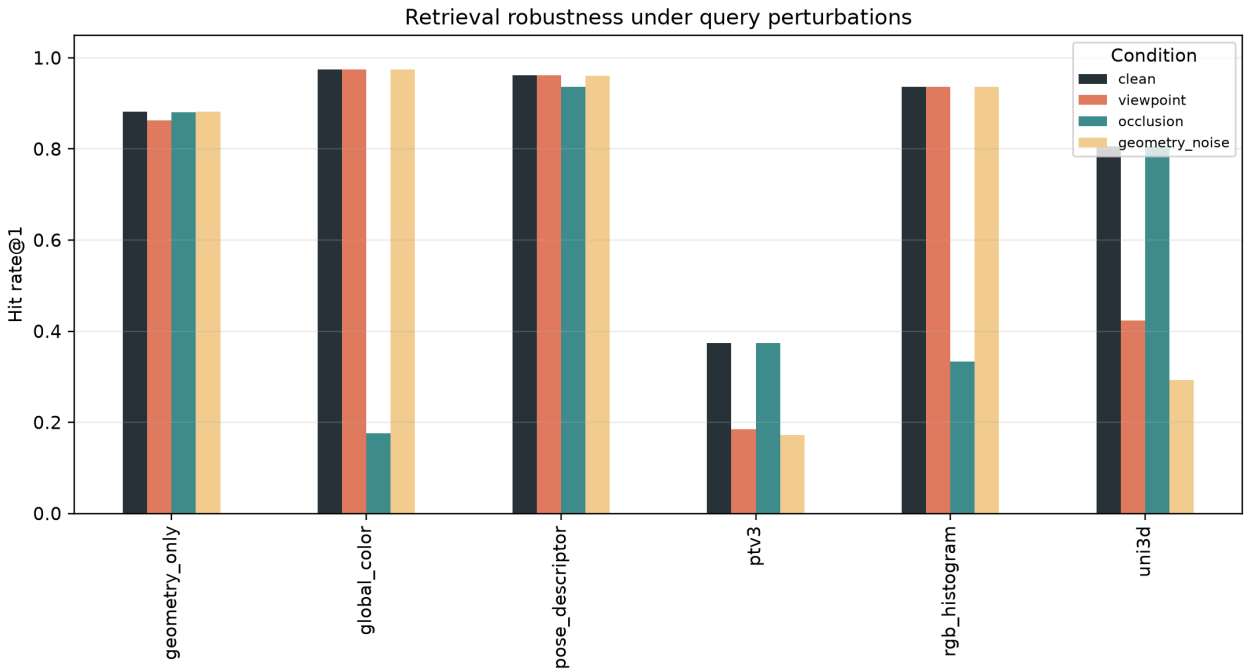
The focused subset contains 704 episodes across eight tasks. The subset results are stored in `retrieval_all/v2_multitask_subset8`. On this subset, the Top-1 accuracies were 0.9744 for global_color, 0.9602 for pose_descriptor, 0.9361 for rgb_histogram, 0.8807 for geometry_only, 0.7656 for Uni3D, 0.6435 for PTv3, and 0.1406 for random. The ordering reinforces the conclusion that appearance and task-correlated color are strong signals in this benchmark.



Subset-8 Top-1 comparison

3.3 Robustness

The robustness evaluation is stored in `robustness/v2_multitask_subset8/summary_metrics.csv` and `robustness/v2_multitask_subset8/evaluation.json`. It evaluates all six non-random retrieval representations under viewpoint, occlusion, and geometry noise perturbations. Since the perturbations affect queries only, the results measure sensitivity of the query embedding and ranking function rather than a change in the database.



Robustness comparison

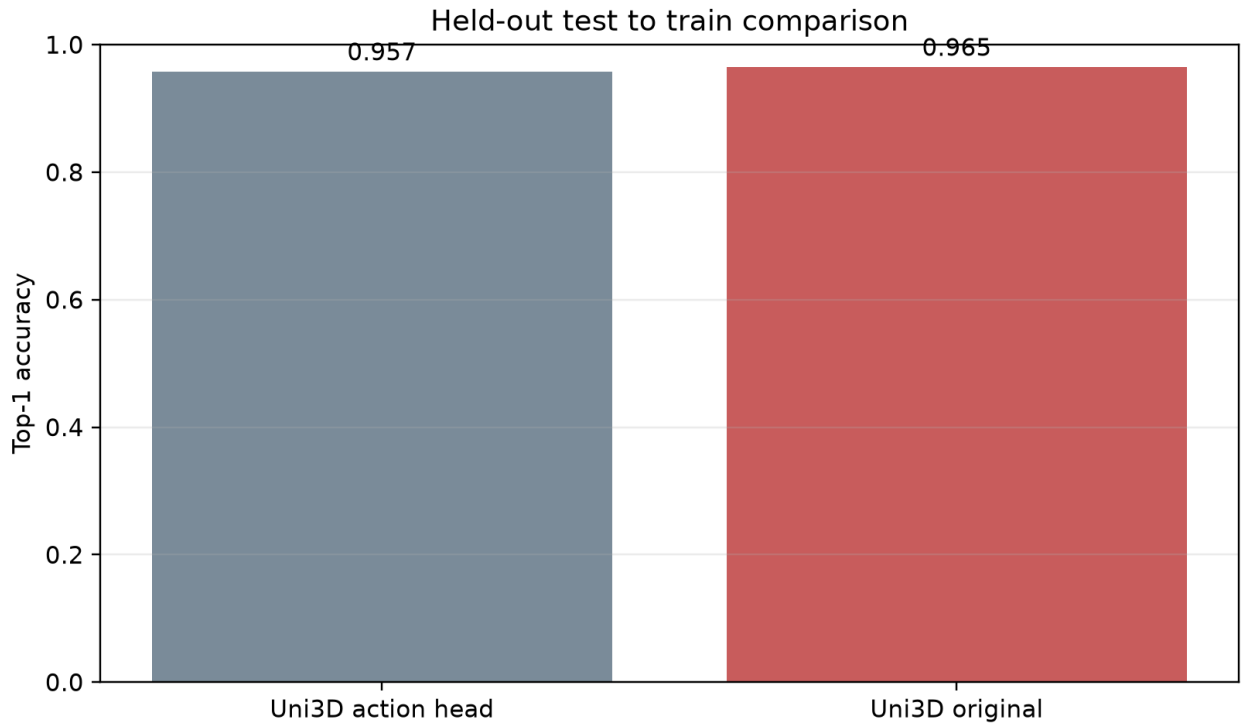
The robustness experiment is evidence about controlled perturbation behavior, not proof of generalization to every possible object geometry or real-world occlusion pattern.

3.4 Held-Out Projection Head

The held-out comparison uses the test split as queries and the train split as candidates. The split contains 422 train episodes and 141 test episodes, with zero overlap.

Method	Top-1 accuracy	Precision@2	Hit rate@2
Uni3D original	0.9645	0.9433	0.9858
Uni3D action head	0.9574	0.9716	0.9929

The action head is slightly worse at Top-1 on held-out data, although it has a higher Precision@2 and Hit rate@2. The earlier 0.9787 Top-1 value was obtained when evaluating all 704 episodes and is therefore reported only as an in-sample diagnostic, not as the primary generalization result.



Held-out projection head comparison

3.5 Offline Downstream Proxy

The downstream experiment in `downstream/v2_multitask_full` evaluates retrieved trajectory transfer offline. It is useful as a bridge from retrieval to downstream use, but it does not execute a learned policy in the simulator.

3.6 Simulator Integration Pilot

The RLBench/CoppeliaSim pilot successfully initialized the simulator in headless mode and executed the replay harness. Four available ReachTarget episodes were checked. The stored trajectories contained derived joint-position sequences, not explicit ground-truth action arrays; the pilot therefore recorded 0/4 successes. This result diagnoses an action/state and initial-scene mismatch and must not be interpreted as evidence that Uni3D or PTv3 fails at learned planning.

4. Discussion and Conclusions

4.1 Main Findings

First, 3D geometric retrieval is useful: geometry-only, Uni3D, and PTv3 all perform above random retrieval. Second, the tested color and pose baselines are stronger than the learned 3D representations in this task-group evaluation. Third, Uni3D is stronger than PTv3 in both the full and subset results, but it does not dominate the color baselines. Fourth, the frozen trajectory-state projection head does not improve held-out Top-1 accuracy over original Uni3D.

These results provide a meaningful negative finding: a modern pretrained 3D backbone is not automatically the limiting or best retrieval signal when task identity is strongly correlated with appearance and color.

4.2 Limitations and Future Work

- Relevance is defined primarily by task-name groups, not by downstream action

compatibility or measured execution success.

- The robustness conditions are synthetic query perturbations.

- The action-aware head predicts a trajectory-state signature, not action

sequences.

- The simulator pilot uses derived joint-position replay because explicit action

arrays are absent.

- No Flow Matching planner or other learned action-generating policy was trained.

- Future work should collect explicit actions, train a trajectory/action decoder,

define action-aware relevance, and evaluate independently generated scene variations.

4.3 Conclusion

The project answers the central retrieval question positively but with an important qualification. Pretrained 3D representations provide a meaningful signal for retrieving manipulation experiences, yet the signal is not universally more robust or more accurate than simple appearance-aware baselines. Uni3D is the stronger of the two tested learned 3D encoders, while PTv3 provides a useful contrasting modern backbone. The completed evidence supports retrieval as a viable component of retrieval-augmented robotics and identifies representation-task alignment as the central open issue.

Appendix A: Machine-Readable Evidence

Machine-readable evidence means the JSON and CSV files produced directly by the evaluation scripts. They preserve exact per-query records, aggregate metrics, configuration metadata, and split information so that the tables and figures in this report can be checked or regenerated programmatically. They support, rather than replace, the human-readable analysis.

Evidence	Path relative to the project/results package	Cluster source path	Purpose
Full retrieval summary	`RAG_for_Robotics_outputs/evaluation/retrieval_full/v2_multitask_full/summary_metrics.csv`	`/cs/labs/raananf/ellorw.nir/3d_cv_dl/RAG_for_Robotics_outputs/evaluation/retrieval_full/v2_multitask_full/summary_metrics.csv`	Aggregate results for all 19 tasks and all methods
Full retrieval details	`RAG_for_Robotics_outputs/evaluation/retrieval_full/v2_multitask_full/per_query_metrics.csv` and `evaluation.json`	Same directory as above	Per-query rankings and run metadata
Robustness summary	`RAG_for_Robotics_outputs/evaluation/robustness/v2_multitask_subset8/summary_metrics.csv`	`/cs/labs/raananf/ellorw.nir/3d_cv_dl/RAG_for_Robotics_outputs/evaluation/robustness/v2_multitask_subset8/summary_metrics.csv`	Viewpoint, occlusion, and geometry-noise results
Held-out projection evaluation	`RAG_for_Robotics_outputs/evaluation/action_head/uni3d_subset8_heldout_final/`	`/cs/labs/raananf/ellorw.nir/3d_cv_dl/RAG_for_Robotics_outputs/evaluation/action_head/uni3d_subset8_heldout_final/`	Test-to-train comparison of original Uni3D and action head
Downstream proxy	`RAG_for_Robotics_outputs/evaluation/downstream/v2_multitask_full/`	`/cs/labs/raananf/ellorw.nir/3d_cv_dl/RAG_for_Robotics_outputs/evaluation/downstream/v2_multitask_full/`	Offline nearest-trajectory transfer evidence
Simulator pilot	`RAG_for_Robotics_outputs/evaluation/planning_pilot/reach_target/`	`/cs/labs/raananf/ellorw.nir/3d_cv_dl/RAG_for_Robotics_outputs/evaluation/planning_pilot/reach_target/`	Guarded RLBench replay result and limitations
Reproducibility snapshot	`RAG_for_Robotics_outputs/evaluation/reproducibility/environment.txt` and `checkpoint_sha256.txt`	`/cs/labs/raananf/ellorw.nir/3d_cv_dl/RAG_for_Robotics_outputs/evaluation/reproducibility/`	Versions, node information, commits, and checkpoint hashes
Dataset metadata	`eval_dataset_metadata/`	`/cs/labs/raananf/ellorw.nir/3d_cv_dl/eval_dataset_metadata/`	Manifest metadata, splits, and SHA-256 checksums

The code that creates these artifacts is organized as follows: retrieval results are produced by `scripts/evaluate_retrieval.py`, robustness by `scripts/evaluate_robustness.py`, projection-head results by `scripts/train_action_aware_projection.py` and `scripts/evaluate_projected_embeddings.py`, the downstream proxy by

`scripts/evaluate_downstream_proxy.py`, and figures by `scripts/plot_report_figures.py`. Reproduction instructions are provided in `README.md`, `ENVIRONMENT_REPRODUCTION.md`, `requirements.txt`, and `requirements-cluster.txt`.

Appendix B: Mathematical Definitions

Let `e_q` denote a query episode and let `D = {e_1, ..., e_N}` denote the candidate database. An encoder with parameters `theta` maps the episode observation `X_e` to an embedding:

$$z_e = f_\theta(X_e), \qquad z_e \in \mathbb{R}^d$$

The ranking score is cosine similarity:

$$s(e_q, e_i) = \frac{z_q^\top z_i}{\|z_q\|_2 \|z_i\|_2}, \qquad R_k(e_q) = \operatorname{argsort}_i s(e_q, e_i)[:k]$$

Here `R_k(e_q)` is the ordered top-k retrieval list. If `rel(q, i)` is one when candidate `i` belongs to the same task relevance group as query `q`, then the principal metrics are:

$$\text{Precision@}k = \frac{1}{k} \sum_{i \in R_k(q)} \text{rel}(q, i)$$

$$\text{Recall@}k = \frac{\sum_{i \in R_k(q)} \text{rel}(q, i)}{\sum_{i \in D} \text{rel}(q, i)}$$

$$\text{HitRate@}k = \mathbf{1}\left[\sum_{i \in R_k(q)} \text{rel}(q, i) > 0\right], \qquad \text{MRR} = \frac{1}{\text{rank}_q}$$

Mean Average Precision and NDCG average rank-sensitive relevance across queries. For the action-aware projection head, the frozen Uni3D embedding is mapped to a trajectory signature by a trainable MLP:

$$h_\phi(z_e) = \text{MLP}_\phi(z_e), \qquad \mathcal{L}(\phi) = \frac{1}{|\mathcal{T}|} \sum_{e \in \mathcal{T}} \|h_\phi(z_e) - a_e\|_2^2$$

`a_e` is the trajectory-state signature derived from the episode, `phi` are the MLP parameters, and the Uni3D parameters `theta` remain frozen. The held- out protocol trains on 422 episodes and evaluates queries from 141 disjoint test episodes against the training candidate set.

References

1. Stephen James et al. “RLBench: The Robot Learning Benchmark & Learning Environment.” arXiv:1909.12271, 2019. https://arxiv.org/abs/1909.12271
2. Ajay Mandlekar et al. “What Matters in Learning from Offline Human Demonstrations for Robot Manipulation.” arXiv:2108.03298, 2021. https://arxiv.org/abs/2108.03298
3. Ajay Mandlekar et al. “MimicGen: A Data Generation System for Scalable Robot Learning using Human Demonstrations.” arXiv:2310.17596, 2023. https://arxiv.org/abs/2310.17596

4. So Kuroki et al. “Multi-Agent Behavior Retrieval: Retrieval-Augmented Policy

Training for Cooperative Push Manipulation by Mobile Robots.” arXiv:2312.02008, 2023. https://arxiv.org/abs/2312.02008

5. I-Chun Arthur Liu et al. “Distilling Motion Planner Augmented Policies into

Visual Control Policies for Robot Manipulation.” CoRL, 2022. https://proceedings.mlr.press/v164/liu22b.html

6. Junsheng Zhou et al. “Uni3D: Exploring Unified 3D Representation at Scale.”

arXiv:2310.06773, 2023. https://arxiv.org/abs/2310.06773

7. Xiaoyang Wu et al. “Point Transformer V3: Simpler, Faster, Stronger.” CVPR,

2024, pp. 4840-4851. https://openaccess.thecvf.com/content/CVPR2024/html/Wu_Point_Transformer_V3_Simpler_Faster_Stronger_CVPR_2024_paper.html